CHAPTER 9

Higher-Ranked Trait Bounds

Every lifetime you have met so far was a *parameter*. In the signature of `longest` from Chapter 1,

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str
```

the caller decides what `'a` is: at each call site the compiler picks the region that makes that call work, and the body of `longest` has to be correct for whichever region the caller picked. That arrangement is perfect for functions that receive borrowed data. It breaks down the moment a function needs to *hand out* a borrow of its own: a buffer it allocated, a value it owns, a line it is reading. No caller can name a lifetime that exists only inside someone else's body.

Callbacks are where this shows up first. A function that reads lines and passes each one to a closure needs the closure to accept a borrow that the function creates, uses and drops. The closure's type must therefore promise more than "I accept a `&str` of some lifetime you agree on with my caller". It must promise "I accept a `&str` of *any* lifetime, including ones that do not exist yet". Rust writes that promise `for<'a>`, and a bound that contains it is a *higher-ranked trait bound* (HRTB).

You have been using higher-ranked bounds for as long as you have written `F: Fn(&str) -> usize`: the compiler inserts `for<'a>` for you. This chapter makes the quantifier visible, shows when you must write it yourself, and teaches you to read the small family of error messages that come from it, messages that otherwise look like they were written in a different language: "implementation of `Fn` is not general enough", "one type is more general than the other".

In this chapter you will learn:

- why a lifetime parameter can never be the lifetime of a borrow of a local variable, and how `for<'a>` moves the choice of lifetime from the caller to the callee
- where the compiler already writes `for<'a>` for you
- the three situations in which you must write it yourself: relationships that `Fn` sugar cannot express, bounds on borrowed types, and traits with a lifetime parameter (the `Deserialize<'de>` pattern)
- why higher-ranked types are "more general", and how to read the errors that say so
- why closures that return references misbehave, and three ways to fix them
- how `AsyncFn` (Rust 1.85) solves async callbacks that borrow their argument
- what early-bound and late-bound lifetimes are, and why a redundant bound can break a call
- the current limits of `for<..>` on stable Rust

9.1 Who picks the lifetime?

Here is a function that creates a string and lends it to a callback. The bound on `F` uses a lifetime parameter of `with_local`, the most natural thing to write if you have only ever seen lifetimes as parameters (`code/compile_fail/ch09_caller_picks.rs`):

```
fn with_local<'a, F>(f: F) -> usize
where
    F: Fn(&'a str) -> usize,
{
    let local = String::from("created inside");
    f(&local)
}
```

The compiler rejects it. The diagnostic continues with a note that points into the standard library's definition of `Fn` (it only says that the requirement comes from the `Fn` bound); the first part is the one that matters:

```
error[E0597]: `local` does not live long enough
  --> compile_fail/ch09_caller_picks.rs:11:7
   |
 6 | fn with_local<'a, F>(f: F) -> usize
   |               -- lifetime `'a` defined here
...
10 |     let local = String::from("created inside");
   |         ----- binding `local` declared here
11 |     f(&local)
   |     --^^^^^^-
   |     | |
   |     | borrowed value does not live long enough
   |     argument requires that `local` is borrowed for `'a`
12 | }
   | - `local` dropped here while still borrowed
```

Read the labels from top to bottom. "lifetime `'a` defined here" points at the parameter list of `with_local`: `'a` belongs to the function's signature, so it is chosen by whoever calls `with_local`. "argument requires that `local` is borrowed for `'a`" points at `f(&local)`: because `F` only accepts `&'a str`, the borrow `&local` must last for the whole of `'a`. Finally, "`local` dropped here while still borrowed" points at the closing brace: `local` dies at the end of the body, but `'a` is still going.

Why is `'a` "still going" at the end of the body? Think about what the caller knows. When it calls `with_local::<'a, F>(f)`, it has to pick `'a` *before* the call starts, from regions it can see: regions of its own body. Whatever it picks, the callee must be correct for it, and the callee cannot rule out that the caller picked a region that includes the whole call and more. From inside `with_local`, then, `'a` is a region that contains every point of the body, from the first line to the closing brace. The borrow checker treats every lifetime parameter of the function it is checking this way (rustc calls them *universal regions*): they outlive the body. A borrow of a local variable, which ends before the body does, can never last that long.

This is the key fact of the chapter:

> **IMPORTANT**
> Every lifetime parameter of a function outlives the function's body. No borrow of a value that the function creates or owns can have the lifetime of one of its parameters.

What `with_local` actually needs is a callback that accepts a borrow of *any* lifetime, so that at the call `f(&local)` the lifetime can be the short region of that one borrow. That is exactly what `for<'a>` says (`code/examples/ch09_callee_picks.rs`):

```
fn with_local<F>(f: F) -> usize
where
    F: for<'a> Fn(&'a str) -> usize, // the same as `F: Fn(&str) -> usize`
{
    let local = String::from("created inside");
    f(&local) + f("a literal works too")
}
```

This version compiles, and the second call shows that "any lifetime" includes `'static` too:

```
33
```

program point	local is alive	F: Fn(&'a str) 'a picked by the caller	F: for<'a> Fn(&'a str) 'a picked at each call of f
caller: with_local(...) begins		'a	
let local = String::from(...)	local	'a	
f(&local)	local	'a: &local must last to the end	'a: just this call
end of body: local is dropped	dropped	'a still live: error E0597	
caller: with_local(...) returns		'a	

Who picks the lifetime. With a lifetime parameter, the caller picks `'a` before the call, so it covers the whole call and the borrow of `local` would have to outlive `local`. With `for<'a>`, a fresh `'a` is picked at each call of `f`, as short as that call.

Reading `for<'a>`

Read `for<'a>` as "for all `'a`". The two signatures say:

- `fn with_local<'a, F>(f: F) where F: Fn(&'a str) -> usize` — "for some `'a` that my caller picks, `f` accepts `&'a str`".
- `fn with_local<F>(f: F) where F: for<'a> Fn(&'a str) -> usize` — "`f` accepts `&'a str` for every `'a`; I will pick a suitable one each time I call it".

The difference is *where the quantifier sits*. In the first signature, `'a` is quantified at the level of the function: the function is generic over `'a`, and the caller instantiates it. In the second, the quantifier sits *inside the bound*: the function is generic over `F`, and `F` itself must be generic over `'a`. A function whose argument must itself be polymorphic is called *higher-rank* (rank 2) in type theory, and that is

where the name of the feature comes from. Only lifetimes can be quantified this way on stable Rust (see *The limits of* `for<..>` below).

Nothing about `for<'a>` makes anything live longer, and it does not mean `'static`. It is a stronger requirement on the *callback*: a closure that accepts only `&'static str` does not satisfy it, while `|s| s.len()` does, because it works for any lifetime.

9.2 Where the compiler writes `for<'a>` for you

You rarely see `for<'a>` because the elided forms of `Fn` bounds, function pointers and trait objects already mean it. The compiler applies the same elision rules as for function signatures (Chapter 3): every elided lifetime in the arguments becomes a fresh lifetime, but bound by the `for<..>` of that very trait or pointer rather than by the enclosing function, and an elided lifetime in the output takes the lifetime of the single input.

You write	The compiler reads
`F: Fn(&str) -> usize`	`F: for<'a> Fn(&'a str) -> usize`
`F: Fn(&str) -> &str`	`F: for<'a> Fn(&'a str) -> &'a str`
`impl Fn(&str) -> usize` (argument)	`impl for<'a> Fn(&'a str) -> usize`
`Box<dyn Fn(&str) -> usize>`	`Box<dyn for<'a> Fn(&'a str) -> usize + 'static>`
`fn(&str) -> &str` (pointer type)	`for<'a> fn(&'a str) -> &'a str`
`F: AsyncFn(&str) -> usize`	`F: for<'a> AsyncFn(&'a str) -> usize`

(The `+ 'static` in the trait-object row is the default object lifetime bound of Chapter 8; it is unrelated to the `for<'a>`.) The following lines from `code/examples/ch09_hr_subtyping.rs` check two of these equivalences: each elided type converts to its explicit higher-ranked form without any coercion.

```
// The elided forms are the same higher-ranked types.
let p: fn(&str) -> &str = str::trim;
let q: for<'a> fn(&'a str) -> &'a str = p;
let b: Box<dyn Fn(&str) -> usize> = Box::new(|s: &str| s.len());
let c: Box<dyn for<'a> Fn(&'a str) -> usize> = b;
println!("{:?} {}", q("  padded  "), c("four"));
```

Two more places produce higher-ranked types without any syntax at all. A *function item* such as `str::trim` or a `fn` you wrote with elided lifetimes is generic over its lifetime at each call, so it converts to the higher-ranked pointer type, as `str::trim` does above. (There is one exception, explained in *Early-bound and late-bound lifetimes* below.) And a *closure written directly as the argument* of a function whose parameter has an `Fn` bound gets its signature from that bound: in `with_local(|s| s.len())`, the closure is checked against `for<'a> Fn(&'a str) -> usize` from the start.

Where to put the binder

`for<..>` can go in two places, and the Rust Reference describes the difference precisely: "Higher-ranked lifetimes may also be specified just before the trait: the only difference is the scope of the lifetime parameter, which extends only to the end of the following trait instead of the whole bound."

```
where F: for<'a> Fn(&'a str) -> usize   // binder before the trait
where for<'a> F: Fn(&'a str) -> usize   // binder before the whole bound
```

For a bound on a type parameter such as `F`, the two are equivalent. The second form is the only one that works when the *bounded type itself* mentions the lifetime, as in `for<'a> &'a C: IntoIterator`, which you will meet shortly.

9.3 When you have to write it yourself

The sugar covers callbacks that take one reference, or that return nothing borrowed. You need to write `for<..>` yourself in three situations.

When the sugar cannot say which input the result comes from

Elision inside `Fn(..)` is elision, so it fails in the same place as for functions: with two reference inputs and a reference output, there is no rule that picks the output's lifetime (`code/compile_fail/ch09_sugar_two_inputs.rs`):

```
pub fn split_head<F>(f: F) -> String
where
    F: Fn(&str, &str) -> &str,
{
    f("key=value", "=").to_string()
}
```

```
error[E0106]: missing lifetime specifier
 --> compile_fail/ch09_sugar_two_inputs.rs:8:26
  |
8 |     F: Fn(&str, &str) -> &str,
  |           ----  ----     ^ expected named lifetime parameter
  |
  = help: this function's return type contains a borrowed value, but the signature does not
say whether it is borrowed from argument 1 or argument 2
  = note: for more information on higher-ranked polymorphism, visit https://doc.rust-lang.
org/nomicon/hrtb.html
help: consider making the bound lifetime-generic with a new `'a` lifetime
  |
8 |     F: for<'a> Fn(&'a str, &'a str) -> &'a str,
  |        +++++++     ++       ++          ++
help: consider making the bound lifetime-generic with a new `'a` lifetime
  |
8 |     for<'a> F: Fn(&'a str, &'a str) -> &'a str,
  |     +++++++        ++       ++          ++
help: consider introducing a named lifetime parameter
  |
6 ~ pub fn split_head<'a, F>(f: F) -> String
7 | where
8 ~     F: Fn(&'a str, &'a str) -> &'a str,
  |
```

This is E0106 again, the error of Chapter 3, with a note pointing to the Rustonomicon's page on higher-ranked polymorphism and three suggestions. The first two make the bound higher-ranked; the third makes `'a` a parameter of `split_head`, which you now know is the wrong kind of lifetime for a callback that will receive borrows of locals.

Look closely at the first two suggestions: they give *both* arguments and the result the same lifetime. That is "candidate B" from Chapter 3, the rule that ties everything together. It always type-checks

the callback, and it moves the cost to the caller of the callback: the result can no longer outlive the *shorter* of the two arguments. If `split_head` keeps the result after the separator is gone, accepting the suggestion verbatim produces a new error (`code/compile_fail/ch09_suggestion_applied.rs`):

```
pub fn split_head<F>(f: F) -> String
where
    F: for<'a> Fn(&'a str, &'a str) -> &'a str, // the compiler's suggestion
{
    let text = String::from("key=value");
    let head;
    {
        let sep = String::from("=");
        head = f(&text, &sep);
    }
    head.to_string()
}
```

```
error[E0597]: `sep` does not live long enough
  --> compile_fail/ch09_suggestion_applied.rs:14:25
   |
13 |         let sep = String::from("=");
   |             --- binding `sep` declared here
14 |         head = f(&text, &sep);
   |                         ^^^^ borrowed value does not live long enough
15 |     }
   |     - `sep` dropped here while still borrowed
16 |     head.to_string()
   |     ---- borrow later used here
```

The bound you want says that the result borrows from the *text* only (`code/examples/ch09_relation.rs`):

```
fn split_head<F>(f: F) -> String
where
    F: for<'t> Fn(&'t str, &str) -> &'t str,
{
    let text = String::from("key=value");
    let head;
    {
        let sep = String::from("=");
        head = f(&text, &sep); // the result borrows `text`, not `sep`
    } // `sep` is dropped here; `head` is still valid
    head.to_string()
}
```

In `for<'t> Fn(&'t str, &str) -> &'t str` the binder introduces `'t` , and the second `&str` is elided *inside* the binder, so it gets a fresh bound lifetime of its own: the bound reads "for all `'t` and all `'s` , `F: Fn(&'t str, &'s str) -> &'t str` ". You can mix named and elided lifetimes this way, exactly as in a function signature. The closure that the caller passes gets this signature from the bound, and its body, which returns a piece of `text` , satisfies it:

```
let head = split_head(|text, sep| text.split(sep).next().unwrap_or(text));
```

```
key
```

When the bounded type contains the lifetime

Suppose you want a function that accepts *any* collection of `i32` (a `Vec` , an array, a `BTreeSet`) and iterates over it twice. You cannot write `C: IntoIterator<Item = i32>` , because iterating a collection by value consumes it. What you want is "a shared borrow of `C` can be iterated, yielding `&i32` ". A borrow

has a lifetime, so the bound has to mention one, and because the function owns `c` and borrows it inside its body, that lifetime must be higher-ranked (`code/examples/ch09_borrowed_iter.rs`):

```
fn stats<C>(c: C) -> (i32, usize)
where
    for<'a> &'a C: IntoIterator<Item = &'a i32>,
{
    let sum = (&c).into_iter().sum();
    let count = (&c).into_iter().count();
    (sum, count)
}
```

Here the binder must come before the whole bound, because the type being bounded, `&'a C` , contains `'a` . The bound reads "for every lifetime `'a` , a `&'a C` can be turned into an iterator over `&'a i32` ". The standard library has exactly such implementations for `&'a Vec<T>` , `&'a [T; N]` , `&'a BTreeSet<T>` and many more, each one generic over `'a` , so all of these calls work:

```
println!("{:?}", stats(vec![1, 2, 3]));
println!("{:?}", stats([10, 20]));
println!("{:?}", stats(BTreeSet::from([5, 6])));
println!("{}", sum_all(&vec![1, 2, 3]));
```

```
(6, 3)
(30, 2)
(11, 2)
6
```

Replace the binder by a lifetime parameter and you are back to the problem of the first section (`code/compile_fail/ch09_borrowed_iter_named.rs`):

```
pub fn stats<'x, C: 'x>(c: C) -> (i32, usize)
where
    &'x C: IntoIterator<Item = &'x i32>,
{
    let sum = (&c).into_iter().sum();
    let count = (&c).into_iter().count();
    (sum, count)
}
```

The compiler reports the same error twice, once for each `(&c)` ; here is the first:

```
error[E0597]: `c` does not live long enough
  --> compile_fail/ch09_borrowed_iter_named.rs:10:15
   |
 6 | pub fn stats<'x, C: 'x>(c: C) -> (i32, usize)
   |              --         - binding `c` declared here
   |              |
   |              lifetime `'x` defined here
...
10 |     let sum = (&c).into_iter().sum();
   |               ^^^^------------
   |               |
   |               borrowed value does not live long enough
   |               argument requires that `c` is borrowed for `'x`
...
13 | }
   | - `c` dropped here while still borrowed
```

`'x` is picked by the caller and outlives the body; `c` is a local of the body (a parameter passed by value is a local like any other); a borrow of `c` cannot last for `'x` . (The `C: 'x` bound is in the program only

to keep the diagnostic short: without it, rustc additionally reports that `C` itself may not live as long as `'x`, error E0309.)

Contrast this with the last function of the example, which takes the collection *by reference*:

```
// When the caller lends the collection, a named lifetime is enough.
fn sum_all<'x, C>(c: &'x C) -> i32
where
    &'x C: IntoIterator<Item = &'x i32>,
{
    c.into_iter().sum()
}
```

Here a lifetime parameter is right: the caller lends the collection for `'x`, so the borrow already exists before the call and lasts for all of it. This gives a practical rule:

> **TIP**
>
> Use a lifetime parameter when the *caller* lends the data. Use `for<'a>` when the *function* creates or owns the data and lends it out itself: to a callback, or through a bound on `&'a T`.

When a trait has a lifetime parameter

Some traits are parameterized by a lifetime because their methods produce values that may borrow from an input. Here is a small one, modelled on the most famous example of the pattern, serde's `Deserialize<'de>` (`code/examples/ch09_trait_param.rs`):

```
/// Parses a value from text that lives for `'src`.
trait Parse<'src>: Sized {
    fn parse(src: &'src str) -> Option<Self>;
}

// A borrowing implementation: the result points into the source text.
impl<'src> Parse<'src> for &'src str {
    fn parse(src: &'src str) -> Option<Self> {
        src.split_whitespace().next()
    }
}

// An owning implementation: it works for any source lifetime.
impl<'src> Parse<'src> for u32 {
    fn parse(src: &'src str) -> Option<Self> {
        src.trim().parse().ok()
    }
}
```

`Parse<'src>` means "can be parsed from text that lives for `'src`". The implementation for `&'src str` is *borrowing*: it returns a slice of the source, so it exists only for the matching `'src` (`&'x str` implements `Parse<'x>` and nothing else). The implementation for `u32` is *owning*: it implements `Parse<'src>` for every `'src`.

There are two ways to bound such a trait, and they correspond exactly to the two kinds of lifetime in this chapter:

```
/// The caller provides the text, so the result may borrow from it.
fn parse_from<'src, T: Parse<'src>>(src: &'src str) -> Option<T> {
    T::parse(src)
}
```

```
/// The text is a local buffer, so `T` must be parseable from a borrow of any
/// lifetime: in practice, `T` cannot borrow from it.
fn parse_local<T>(bytes: &[u8]) -> Option<T>
where
    T: for<'src> Parse<'src>,
{
    let text = String::from_utf8_lossy(bytes).into_owned();
    T::parse(&text)
}
```

`parse_from` lets the caller provide the text, so it uses a lifetime parameter and accepts borrowing types. `parse_local` parses a buffer it creates itself, so it needs `T: for<'src> Parse<'src>`: "`T` can be parsed from text of any lifetime". Both work for `u32`, and `parse_from` also works for `&str`:

```
let text = String::from("borrowed words");
let word: Option<&str> = parse_from(&text);
let n: Option<u32> = parse_from(" 7 ");
let m: Option<u32> = parse_local(b" 42 ");
println!("{word:?} {n:?} {m:?}");
```

```
Some("borrowed") Some(7) Some(42)
```

Asking `parse_local` for a `&str` cannot work: the result would point into a buffer that `parse_local` drops before returning. The compiler does not phrase it that way, though (`code/compile_fail/ch09_trait_param_borrow.rs`):

```
pub fn caller() {
    let word: Option<&str> = parse_local(b"words");
    println!("{word:?}");
}
```

```
error: implementation of `Parse` is not general enough
  --> compile_fail/ch09_trait_param_borrow.rs:25:30
   |
25 |     let word: Option<&str> = parse_local(b"words");
   |                              ^^^^^^^^^^^^^^^^^^^^^ implementation of `Parse` is not
general enough
   |
   = note: `Parse<'0>` would have to be implemented for the type `&str`, for any lifetime
`'0`...
   = note: ...but `Parse<'1>` is actually implemented for the type `&'1 str`, for some
specific lifetime `'1`
```

This is the message you will see most often with higher-ranked bounds, so it is worth reading slowly. The first note states the requirement: "`Parse<'0>` would have to be implemented for the type `&str`, for any lifetime `'0`". That is the bound `for<'src> Parse<'src>` with the bound lifetime renamed `'0`. The second note states what exists: "`Parse<'1>` is actually implemented for the type `&'1 str`, for some specific lifetime `'1`". The implementation for `&'1 str` covers exactly one lifetime, `'1`, so it is "not general enough" to cover all of them. In rustc's notes, *for any lifetime* marks what the bound requires and *for some specific lifetime* marks what you have.

This is precisely serde's design. A `T: Deserialize<'de>` may borrow from input that lives for `'de`, which is why `serde_json::from_str` can deserialize into `&'a str` fields; a `T: DeserializeOwned` cannot borrow, which is what you need when the input is a buffer the deserializer owns, as in `serde_json::from_reader`.

In serde's own words (https://serde.rs/lifetimes.html), "the `DeserializeOwned` trait is equivalent to the higher-rank trait bound `for<'de> Deserialize<'de>`." The signatures in serde_json (not compiled here: the book's code has no dependencies) are:

```
pub fn from_str<'a, T>(s: &'a str) -> Result<T>
where
    T: Deserialize<'a>,

pub fn from_reader<R, T>(rdr: R) -> Result<T>
where
    R: Read,
    T: DeserializeOwned,
```

`from_str` is `parse_from`; `from_reader` is `parse_local`.

9.4 Higher-ranked types are more general

A higher-ranked type promises something for *every* lifetime, so it can be used wherever the promise is needed for *one* lifetime. For function pointers this is subtyping, and the Reference describes it in one sentence: higher-ranked function pointers "are subtypes of types that are given by substitutions of the higher-ranked lifetimes." Substituting `'static` for `'a` in `for<'a> fn(&'a str) -> usize` gives `fn(&'static str) -> usize`, so the first converts to the second (`code/examples/ch09_hr_subtyping.rs`):

```
// `len` works for every lifetime...
let any: for<'a> fn(&'a str) -> usize = len;
// ...so it can be used where only one specific lifetime is required.
let only_static: fn(&'static str) -> usize = any;
let local = String::from("hello");
println!("{} {}", any(&local), only_static("hi"));
```

```
5 2
"padded" 4
```

The other direction fails, because a function that accepts only `&'static str` cannot accept a borrow of a local (`code/compile_fail/ch09_hr_subtyping.rs`):

```
fn static_only(s: &'static str) -> usize {
    s.len()
}

pub fn caller() {
    let f: fn(&'static str) -> usize = static_only;
    let any: for<'a> fn(&'a str) -> usize = f;
    println!("{}", any("hi"));
}
```

```
error[E0308]: mismatched types
  --> compile_fail/ch09_hr_subtyping.rs:12:45
   |
12 |     let any: for<'a> fn(&'a str) -> usize = f;
   |              ----------------------------   ^ one type is more general than the other
   |              |
   |              expected due to this
   |
   = note: expected fn pointer `for<'a> fn(&'a _) -> _`
              found fn pointer `fn(&'static _) -> _`
```

"one type is more general than the other" is how E0308 (mismatched types) reads when the two types differ only in how their lifetimes are quantified. The notes show both types with the irrelevant parts replaced by `_`: the expected one is higher-ranked (`for<'a> fn(&'a _) -> _`), the one you have fixes the lifetime (`fn(&'static _) -> _`).

Closures whose signature was fixed too early

The same mismatch appears with closures, with a twist: whether a closure's signature is higher-ranked depends on how its parameter types were inferred. In this program, the closure's parameter has no annotation, and its first use fixes the parameter type (`code/compile_fail/ch09_not_general_closure.rs`):

```
pub fn caller() -> usize {
    let count = |s| str::len(s);
    count("literal"); // fixes the parameter type: `&'static str`
    call_on_local(count)
}
```

The error is reported twice, for `Fn` and for its supertrait `FnOnce`; the first copy is:

```
error: implementation of `Fn` is not general enough
  --> compile_fail/ch09_not_general_closure.rs:13:5
   |
13 |     call_on_local(count)
   |     ^^^^^^^^^^^^^^^^^^^^ implementation of `Fn` is not general enough
   |
   = note: closure with signature `fn(&'2 str) -> usize` must implement `Fn<(&'1 str,)>`,
for any lifetime `'1`...
   = note: ...but it actually implements `Fn<(&'2 str,)>`, for some specific lifetime `'2`
help: consider adding an explicit type annotation to the closure's argument
   |
11 |     let count = |s: &str| str::len(s);
   |                  ++++++
```

The first call, `count("literal")`, made the parameter type `&'2 str` for one specific lifetime `'2` (the closure now has "signature `fn(&'2 str) -> usize`"). When the closure is later passed to `call_on_local`, whose bound is `Fn(&str) -> usize`, which is `for<'1> Fn(&'1 str) -> usize`, it is not general enough. A closure gets a higher-ranked signature when, *at the point where it is written*, either its parameters are annotated with reference types or it is checked against an expected `Fn` bound. The compiler's help suggests the first: write `|s: &str|`.

> **NOTE**
>
> "implementation of `Fn` is not general enough" and "one type is more general than the other" are the same diagnosis in two forms. The first appears when a trait must be implemented for every lifetime; the second when two *types* must be equal. In both, the fix is to make the thing you pass as general as the thing that is required.

9.5 Closures that return references

Annotating the parameter makes a closure's *argument* higher-ranked. It does not help with the *result*. A closure that returns part of its argument, stored in a `let` binding, is rejected, and so is the same closure with an annotated return type (`code/compile_fail/ch09_closure_return.rs`):

```
pub fn caller() {
    let first_word = |s: &str| s.split(' ').next().unwrap_or("");
    let annotated = |s: &str| -> &str { s.split(' ').next().unwrap_or("") };
    let text = String::from("hello world");
    println!("{} {}", first_word(&text), annotated(&text));
}
```

```
error: lifetime may not live long enough
 --> compile_fail/ch09_closure_return.rs:8:32
  |
8 |     let first_word = |s: &str| s.split(' ').next().unwrap_or("");
  |                          -   - ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ returning this value
requires that `'1` must outlive `'2`
  |                          |   |
  |                          |   return type of closure is &'2 str
  |                          let's call the lifetime of this reference `'1`

error: lifetime may not live long enough
 --> compile_fail/ch09_closure_return.rs:9:41
  |
9 |     let annotated = |s: &str| -> &str { s.split(' ').next().unwrap_or("") };
  |                          -        -     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ returning this
value requires that `'1` must outlive `'2`
  |                          |        |
  |                          |        let's call the lifetime of this reference `'2`
  |                          let's call the lifetime of this reference `'1`
```

In the first error, `'1` is the lifetime of the argument, which is higher-ranked because of the annotation `&str`. The return type, though, was inferred from the body as `&'2 str` for one fixed lifetime `'2` ("return type of closure is `&'2 str`"). Returning `s` would require `'1: '2` for every possible `'1`, which no single `'2` can satisfy. The second error shows that writing `-> &str` does not tie the two together: the function elision rules of Chapter 3 do not apply to closure annotations, so the `&str` in the return type gets a lifetime of its own, `'2`, unrelated to the argument's `'1`.

There are three reliable fixes (`code/examples/ch09_closure_fixes.rs`):

```
/// Identity function whose only job is to give the closure an expected
/// higher-ranked signature.
fn hr<F: Fn(&str) -> &str>(f: F) -> F {
    f
}

fn first_word(s: &str) -> &str {
    s.split(' ').next().unwrap_or("")
}

fn apply<F: Fn(&str) -> &str>(f: F, s: &str) -> usize {
    f(s).len()
}
```

```
let text = String::from("hello world");
// 1. Pass the closure directly to a function with a higher-ranked bound.
let n = apply(|s| s.trim(), &text);
// 2. Route a stored closure through an identity function with that bound.
let a = hr(|s| s.split(' ').next().unwrap_or(""));
// 3. Use a fn item: it is generic over the (late-bound) lifetime.
let b = first_word;
println!("{n} {} {}", a(&text), b(&text));
```

```
11 hello hello
```

1. **Pass the closure directly** to a function with a higher-ranked `Fn` bound. The closure is checked against `for<'a> Fn(&'a str) -> &'a str` when it is created, and gets exactly that signature. This is what you do most of the time without noticing.
2. **Route a stored closure through an identity function** whose parameter has the bound you want. `hr` does nothing at run time; its only job is to provide the expected signature at the point where the closure is written.
3. **Use a `fn` item.** A function with elided lifetimes is generic over them at each call, so `first_word` already has the signature you want.

Rust has a syntax that would say this directly, a binder on the closure itself: `for<'a> |s: &'a str| -> &'a str { ... }` (RFC 3216, "closure lifetime binder"). As of Rust 1.99 it is unstable (tracking issue https://github.com/rust-lang/rust/issues/97362); you will see the error it produces on stable at the end of this chapter.

9.6 Async callbacks that borrow

Async code makes the callback problem harder. Suppose `with_local` wants a callback that returns a *future*, and the future needs the borrowed string while it runs. Before Rust 1.85, the natural way to write the bound was to give the future type its own parameter (`code/compile_fail/ch09_async_fut_param.rs` ; the small `block_on` function it uses is shown below):

```
fn with_local<F, Fut>(f: F) -> usize
where
    F: Fn(&str) -> Fut,
    Fut: Future<Output = usize>,
{
    let local = String::from("created inside");
    block_on(f(&local))
}

async fn count_words(s: &str) -> usize {
    s.split_whitespace().count()
}

pub fn caller() -> usize {
    with_local(count_words)
}
```

```
error[E0308]: mismatched types
  --> compile_fail/ch09_async_fut_param.rs:34:5
   |
34 |     with_local(count_words)
   |     ^^^^^^^^^^^^^^^^^^^^^^^ one type is more general than the other
   |
   = note: expected opaque type `impl for<'a> Future<Output = usize>`
              found opaque type `impl Future<Output = usize>`
   = note: distinct uses of `impl Trait` result in different opaque types
note: the lifetime requirement is introduced here
  --> compile_fail/ch09_async_fut_param.rs:22:20
   |
22 |     F: Fn(&str) -> Fut,
   |                    ^^^
```

`F: Fn(&str) -> Fut` is `for<'a> Fn(&'a str) -> Fut`, and the binder covers only the arguments: `Fut` is a type parameter of `with_local`, picked once by the caller, the same for every `'a`. But an

`async fn` returns a future that captures its arguments (Chapter 11), so `count_words(&'a str)` returns a *different type* for each `'a`: a type that mentions `'a`. The note says exactly this: it expected "`impl for<'a> Future<Output = usize>`", a future type that works for all `'a`, and found "`impl Future<Output = usize>`" for one particular `'a`. The note "the lifetime requirement is introduced here" points at `Fut`.

There was no good way to write this bound before 1.85 (the usual workaround was to box the future). Rust 1.85 added the `AsyncFn`, `AsyncFnMut` and `AsyncFnOnce` traits, together with async closures, and they solve exactly this problem; the release announcement says it "has not been possible to properly express higher-ranked function signatures with the `Fn` traits returning a `Future`, but you can write this with the `AsyncFn` traits" (https://blog.rust-lang.org/2025/02/20/Rust-1.85.0/). With `AsyncFn` the future is not a separate parameter: it is an associated type of the trait, and the trait is implemented separately for each argument lifetime, so the future may depend on `'a` (`code/examples/ch09_async_fn.rs`):

```
fn with_local<F>(f: F) -> usize
where
    F: AsyncFn(&str) -> usize,
{
    let local = String::from("created inside");
    block_on(f(&local))
}

async fn count_words(s: &str) -> usize {
    s.split_whitespace().count()
}
```

```
let bonus = 100;
println!("{}", with_local(count_words));
println!("{}", with_local(async |s: &str| s.len() + bonus));
```

```
2
114
```

Both an `async fn` and an async closure (`async |s: &str| ...`, which here also uses the captured variable `bonus`) satisfy `AsyncFn(&str) -> usize`, which is sugar for `for<'a> AsyncFn(&'a str) -> usize` like any other `Fn`-style bound.

The examples need a way to run a future without an async runtime, because the book's code uses only the standard library. This is the whole executor:

```
/// Polls a future to completion on the current thread. Good enough for futures
/// that never wait on anything external, like the ones in this example.
fn block_on<T>(fut: impl Future<Output = T>) -> T {
    let mut fut = pin!(fut);
    let mut cx = Context::from_waker(Waker::noop());
    loop {
        if let Poll::Ready(v) = fut.as_mut().poll(&mut cx) {
            return v;
        }
    }
}
```

`Waker::noop()` (stable since 1.85) is a waker that does nothing: it is enough for futures that never wait for anything external, because such futures complete the first time they are polled. Chapter 11 uses the same `block_on` and goes further into lifetimes in async code.

9.7 Early-bound and late-bound lifetimes

You saw that a function item such as `first_word` can be used where a higher-ranked `Fn` bound is required, because it is generic over its lifetime at each call. That is true for most lifetime parameters, but not all, and the difference occasionally produces a baffling error. Here are two functions that differ only in a bound that looks redundant (`code/compile_fail/ch09_early_bound.rs`):

```
fn apply<F: Fn(&[i32]) -> &i32>(f: F) -> i32 {
    let v = vec![3, 1, 2];
    *f(&v)
}

fn first_late<T>(v: &[T]) -> &T {
    &v[0]
}

fn first_early<'a, T: 'a>(v: &'a [T]) -> &'a T {
    &v[0]
}

pub fn caller() -> i32 {
    apply(first_late) + apply(first_early)
}
```

`T: 'a` is implied by `&'a [T]` (Chapter 1), so writing it changes nothing about which calls of `first_early` are valid. Yet `apply(first_late)` compiles and `apply(first_early)` does not (the error is again reported for `Fn` and for `FnOnce` ; the first copy is shown):

```
error: implementation of `Fn` is not general enough
  --> compile_fail/ch09_early_bound.rs:21:25
   |
21 |     apply(first_late) + apply(first_early)
   |                         ^^^^^^^^^^^^^^^^^^ implementation of `Fn` is not general enough
   |
   = note: `fn(&'2 [i32]) -> &'2 i32 {first_early::<'2, i32>}` must implement `Fn<(&'1
[i32],)>`, for any lifetime `'1`...
   = note: ...but it actually implements `Fn<(&'2 [i32],)>`, for some specific lifetime `'2`
```

The note gives the type of the function item: `{first_early::<'2, i32>}` , with the lifetime *already filled in* with a specific `'2` , like the type parameter `i32` . That is the difference:

- A **late-bound** lifetime is chosen at each call. The function item's type stays generic over it, so `first_late::<i32>` has the higher-ranked signature `for<'a> fn(&'a [i32]) -> &'a i32` and satisfies `Fn(&[i32]) -> &i32` .
- An **early-bound** lifetime is chosen when the function is *named*, like a type parameter, so `first_early::<'2, i32>` works for that one `'2` only.

Which lifetimes are early-bound is a rule of rustc rather than something you declare. According to the rustc development guide (https://rustc-dev-guide.rust-lang.org/early-late-parameters.html), a lifetime parameter is early-bound when it appears in a where clause or bound (even a trivial one such

as `'a: 'a`, or `T: 'a` as here), or when it appears only in the return type; an *implied* `T: 'a` keeps it late-bound. In practice: **do not add `T: 'a` bounds to functions when the argument types already imply them.**

Late-bound lifetimes have one more visible consequence: since they are chosen at each call, you cannot fix them with a turbofish (`code/compile_fail/ch09_late_turbofish.rs`):

```
fn late<'a>(s: &'a str) -> &'a str {
    s
}

pub fn caller() -> &'static str {
    let f = late::<'static>;
    f("literal")
}
```

```
error[E0794]: cannot specify lifetime arguments explicitly if late bound lifetime parameters
are present
  --> compile_fail/ch09_late_turbofish.rs:11:20
   |
11 |     let f = late::<'static>;
   |                    ^^^^^^^
   |
note: the late bound lifetime parameter is introduced here
  --> compile_fail/ch09_late_turbofish.rs:6:9
   |
 6 | fn late<'a>(s: &'a str) -> &'a str {
   |         ^^
```

9.8 The limits of `for<..>`

Higher-ranked bounds on stable Rust 1.99 have a few hard edges. This file collects four of them (`code/compile_fail/ch09_binder_limits.rs`):

```
// 1. A bound lifetime used only in the output.
pub fn output_only<F>(f: F)
where
    F: for<'a> Fn() -> &'a str,
{
    let _ = f;
}

// 2. Two nested binders on the same bound.
pub fn nested<F>(f: F)
where
    for<'a> F: for<'b> Fn(&'a str, &'b str),
{
    let _ = f;
}

// 3. A binder over types.
pub fn over_types<F>(f: F)
where
    F: for<T> Fn(T),
{
    let _ = f;
}

// 4. A binder on a closure.
pub fn closure_binder() -> usize {
    let len = for<'a> |s: &'a str| -> usize { s.len() };
```

```
    len("four")
}
```

```
error[E0316]: nested quantification of lifetimes
  --> compile_fail/ch09_binder_limits.rs:16:16
   |
16 |     for<'a> F: for<'b> Fn(&'a str, &'b str),
   |                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^

error[E0658]: `for<...>` binders for closures are experimental
  --> compile_fail/ch09_binder_limits.rs:31:15
   |
31 |     let len = for<'a> |s: &'a str| -> usize { s.len() };
   |               ^^^^^^^
   |
   = note: see issue #97362 <https://github.com/rust-lang/rust/issues/97362> for more
information
   = help: consider removing `for<...>`

error[E0658]: only lifetime parameters can be used in this context
  --> compile_fail/ch09_binder_limits.rs:24:12
   |
24 |     F: for<T> Fn(T),
   |            ^
   |
   = note: see issue #108185 <https://github.com/rust-lang/rust/issues/108185> for more
information

error[E0582]: binding for associated type `Output` references lifetime `'a`, which does not
appear in the trait input types
 --> compile_fail/ch09_binder_limits.rs:8:24
  |
8 |     F: for<'a> Fn() -> &'a str,
  |                        ^^^^^^^
```

Construct	Result in Rust 1.99	What to do
`for<'a> Fn() -> &'a str`	E0582	The result would be valid for any lifetime the caller wants, which only `'static` data can be: write `Fn() -> &'static str`.
`for<'a> F: for<'b> Fn(..)`	E0316	Use one binder: `F: for<'a, 'b> Fn(&'a str, &'b str)`.
`for<T>` (a type in a binder)	E0658	Unstable (`non_lifetime_binders`, issue #108185).
`for<'a> |s: &'a str| ...`	E0658	Unstable (`closure_lifetime_binder`, issue #97362); see *Closures that return references*.
`for<'a where 'a: 'b>`	not valid syntax	No bounds on bound lifetimes; tracked by the 2026 project goal "Next-Gen Region Constraints".

E0582 deserves a sentence more. In `for<'a> Fn() -> &'a str`, `'a` appears in the output but in none of the inputs, so nothing in a call constrains it: the caller of `f` could pick *any* `'a`. The bound would describe a function that can conjure a reference valid for arbitrarily long, which is what `'static` already says. The same E0582 appears for traits with associated types (`for<'a> Tr<Assoc = &'a str>` where `'a` is not in the trait's own parameters).

The last row has a consequence that you will meet in Chapter 10. A binder ranges over *all* lifetimes, including lifetimes much longer than the types involved, and you cannot restrict it to "all `'a` that `T` outlives". When a bound such as `for<'a> L::Item<'a>: Debug` meets a generic associated type declared

with `where Self: 'a`, the only way for the bound to hold for all `'a` is `L: 'static`, and rustc says so in a note: "due to a current limitation of the type system, this implies a `'static` lifetime". Chapter 10 shows the problem in full and two ways around it.

9.9 Reading higher-ranked errors

You see	It means	Usual fix
"`x` does not live long enough ... argument requires that `x` is borrowed for `'a`", where `'a` is a lifetime parameter of the function	A caller-chosen lifetime is being used for a borrow the function creates	Make the bound higher-ranked (`for<'a>`, or the elided `Fn(&T)` form)
"implementation of `Trait` is not general enough ... for any lifetime `'1` ... for some specific lifetime `'2`"	Something implements the trait for one lifetime; the bound needs all of them	Annotate closure parameters; remove early-binding bounds; use an owning type for `for<'de>`-style bounds
E0308 "one type is more general than the other"	A type with a fixed lifetime where a higher-ranked type is required (fn pointers, or a `Fut` parameter for an async callback)	Use the higher-ranked type; for async callbacks use `AsyncFn`
"lifetime may not live long enough ... returning this value requires that `'1` must outlive `'2`" inside a closure	A stored closure returns a reference to its argument	Pass the closure to a bound, use an identity helper, or use a `fn`
E0106 in an `Fn(..)` bound	Elision inside the sugar failed	Write the binder and say which input the output borrows from
E0582, E0316, E0794, E0658	The limits in the previous section	See the table above

9.10 Common confusions

- "**`for<'a>` means `'static`.**" It means *every* lifetime, the shortest ones included. A callback that only works for `'static` data does *not* satisfy a higher-ranked bound.
- "**I need `for<'a>` for every callback.**" Almost never: `Fn(&T) -> U` already is higher-ranked. You write the binder for two-input relationships, bounds on `&'a T`, and traits with a lifetime parameter.
- "**A lifetime on the struct or `impl` works just as well.**" It is a parameter, picked by whoever creates the value. A struct `Handler<'a, F: Fn(&'a str)>` can never pass a borrow of its own temporary data to `F`.
- "**Higher-ranked bounds keep the closure alive longer.**" They say nothing about how long the closure lives; they constrain which arguments it accepts.
- "**The compiler's suggestion is the fix.**" For E0106 in an `Fn` bound, the suggestion ties all lifetimes together. It compiles the callback, but it may be the wrong contract for the code that calls it.

9.11 Summary

- A lifetime parameter is picked by the caller and outlives the whole body, so it can never be the lifetime of a borrow the function creates.
- `for<'a>` moves the choice to the callee: the bound must hold for every lifetime, and each use picks the one it needs. A bound that contains it is a higher-ranked trait bound.
- `Fn(&T) -> U`, `fn(&T) -> U`, `dyn Fn(&T) -> U` and `AsyncFn(&T) -> U` are already higher-ranked; elision inside them follows the function rules.
- Write `for<..>` yourself to relate one of several inputs to the output, to bound a borrowed type (`for<'a> &'a C: IntoIterator`), and to bound a trait with a lifetime parameter (`for<'de> Deserialize<'de>`, i.e. `DeserializeOwned`).
- Higher-ranked types are more general: they convert to any of their instances, never the other way. "not general enough" and "one type is more general than the other" both mean "you passed something less general than required".
- Closures get higher-ranked argument types from annotations or from an expected bound, but a stored closure cannot return a reference tied to its argument; pass it to a bound, use an identity helper, or use a `fn`.
- `AsyncFn` (Rust 1.85) expresses async callbacks whose futures borrow their arguments, which `F: Fn(&T) -> Fut` cannot.
- Lifetimes that appear in bounds are early-bound and make a function item less general; avoid redundant `T: 'a` bounds on functions.

9.12 Exercises

1. Predict whether this function compiles, then check:
   ```
   fn total<'a, F: Fn(&'a str) -> usize>(f: F, items: &[String]) -> usize { items.iter().map(|s| f(s)).sum() }
   ```
   Fix it in two different ways, and explain which fix gives callers more freedom. *Hint:* one fix ties `items` to `'a`; the other removes `'a` from the signature.
2. Which of these bounds need an explicit `for<..>` to compile, and what would you write? `F: Fn(&str) -> String`, `F: Fn(&str, &str) -> bool`, `F: Fn(&str, usize) -> &str`, `F: Fn(&str, &str) -> &str` (the result comes from the second argument). *Hint:* apply the elision rules of Chapter 3 to the arguments of each bound.
3. Change `parse_local` from `code/examples/ch09_trait_param.rs` so that it can return borrowing types such as `&str`. What must change about who owns the buffer? *Hint:* look at `parse_from`, and at the difference between `from_reader` and `from_str`.
4. `let longer = |a: &str, b: &str| if a.len() >= b.len() { a } else { b };` does not compile. Explain the error with the vocabulary of this chapter, then make it compile with an identity helper. *Hint:* the helper's bound needs a binder with one lifetime shared by both arguments and the result.

5. Write `fn retry<F>(mut f: F) -> Option<usize> where F: AsyncFnMut(&str) -> Option<usize>` that builds a local `String` and calls `f` on it up to three times, returning the first `Some`. Run it with the `block_on` of this chapter and an async closure that counts its calls. *Hint:* `AsyncFnMut` lets the closure mutate captured state; the future returned by each call must be awaited (or blocked on) before the next call.
6. In `code/compile_fail/ch09_early_bound.rs`, rewrite `first_late` with a named lifetime (`fn first_late<'a, T>(v: &'a [T]) -> &'a T`) and check that `apply(first_late)` still compiles. Then add `where T: 'a` to it, and finally remove the bound from `first_early`. Explain all three results with the early-bound / late-bound distinction. *Hint:* the note in the diagnostic shows the function item's type with its lifetime filled in.